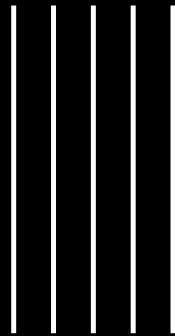


Simulasi Realistis Git & GitHub Team Workflow



Afrizal F.A

Simulasi Realistis Git & GitHub Team Workflow

Latihan tim 1–4 akun: Pull Request, review, CI, conflict,
squash merge, hotfix, parallel work, dan cherry-pick

Afrizal F.A

2026

Kata Pengantar

Buku ini disusun sebagai modul praktik Git & GitHub dengan format simulasi tim yang realistis: ada Tech Lead, developer, reviewer, ticket, Pull Request, CI, konflik, hotfix, sampai cherry-pick ke release branch. Semua nama perusahaan, akun, dan ticket adalah fiksi untuk latihan.

Penjelasan sengaja dibuat singkat dan to the point. Teori hanya secukupnya; sisanya kamu pelajari dengan mengetik command dan mengalami prosesnya sendiri. Kerjakan bab secara berurutan, karena setiap bab dibangun di atas bab sebelumnya.

Target pembaca: siapa pun yang sudah bisa membuka terminal dan ingin siap bekerja dalam tim software modern. Selamat berlatih.

Daftar Isi

BAB 1 – Pendahuluan

BAB 2 – Persiapan dan Repository Awal

BAB 3 – Task Pertama: APP-101

BAB 4 – Review, CI, dan Revisi

BAB 5 – Konflik dan Integrasi

BAB 6 – Squash Merge, Sinkronisasi, dan Hotfix

BAB 7 – Simulasi 3 Akun: Parallel Work

BAB 8 – Simulasi 4 Akun: Dua Reviewer

BAB 9 – Cherry-pick

BAB 10 – Penutup

BAB 1 – Pendahuluan

Kamu bergabung ke tim fiksi Acme App Team yang mengelola repository `acme-profile-app`: aplikasi profil sederhana berbasis JavaScript. Lewat repository ini kamu menjalani siklus kerja tim software modern dari awal sampai selesai.

```
main <- Pull Request <- feature branch <- commits lokal
```

Empat mode latihan

Mode	Akun	Yang dilatih
Pemanasan	1	Branch, commit, push, PR, merge
Realistis	2	Protected main, request changes, approval
Tim asli	3	Parallel work, review silang, konflik alami
Perusahaan	4	Dua reviewer, CODEOWNERS, approval ganda

Pembagian role

Akun	Role	Tanggung jawab
A	Tech Lead / Maintainer	Repo, aturan main, ticket, review akhir, merge
B	Developer 1 / Reviewer 1	Coding, PR, me-review PR orang lain
C	Developer 2 / Reviewer 2	Coding paralel, review silang (mode 3-4)
D	Developer (author)	Mengerjakan task, menangani feedback (mode 4)

Prinsip utama buku ini: developer tidak pernah bekerja langsung di main. Semua perubahan masuk lewat branch dan Pull Request yang direview.

BAB 2 – Persiapan dan Repository Awal

Setup Git

```
git --version
git config --global user.name "Nama Kamu"
git config --global user.email "email-kamu"
```

Multi-akun di satu komputer

Untuk mode 2-4 akun, set identitas per repository (tanpa --global) di setiap folder clone, dan gunakan kredensial/SSH key berbeda per akun.

```
cd acme-profile-app-dev2
git config user.name "Dev C"
git config user.email "devc@example.com"
```

Project awal (Akun A – Tech Lead)

```
acme-profile-app/
|-- package.json          # "test": "node --test"
|-- .github/workflows/ci.yml
|-- src/profile.js
`-- test/profile.test.js
```

```
// src/profile.js
function createProfile(displayName, bio) {
  return { displayName, bio };
}
module.exports = { createProfile };
```

```
// test/profile.test.js (node:test, tanpa dependency)
const { test } = require("node:test");
const assert = require("node:assert/strict");
const { createProfile } = require("../src/profile");

test("creates a profile", () => {
  assert.equal(createProfile("Raka", "dev").displayName, "Raka");
});
```

```
# .github/workflows/ci.yml
name: ci
on: [pull_request, push]
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with: { node-version: 20 }
      - run: npm test
```



```
git init
git add .
git commit -m "chore: initialize profile app"
git branch -M main
git remote add origin <URL-REPOSITORY>
git push -u origin main
```

Aturan branch main

Aturan	Tujuan
Require Pull Request	Tidak ada direct push ke main
Require approval	Minimal 1 reviewer setuju (mode 4: minimal 2)
Require status checks	CI harus hijau sebelum merge
Block force push	History main tidak ditulis ulang

Tambahkan akun B (dan C, D sesuai mode) sebagai collaborator. Jika fitur protection tidak tersedia di plan repositorymu, jalankan aturannya secara disiplin manual.

BAB 3 – Task Pertama: APP-101

Ticket dari Tech Lead

```
APP-101 - Add editable user profile
- displayName dan bio tersimpan
- displayName wajib diisi
- Spasi depan/belakang dibersihkan
- Test lulus
Out of scope: avatar, database, auth
```

Clone dan buat branch (Akun B)

```
git clone <URL-REPOSITORY> && cd acme-profile-app
git switch main
git pull origin main
git switch -c feature/APP-101-editable-profile
```

Branch selalu dibuat dari main yang baru di-pull, bukan snapshot lama di laptop.

Commit 1 - normalisasi

```
function createProfile(displayName, bio = "") {
  return {
    displayName: displayName.trim(),
    bio: bio.trim(),
  };
}
```

```
git status && git diff
npm test
git add src/profile.js
git commit -m "feat: normalize profile fields"
```

Commit 2 - validation + test

```
const cleanName = displayName.trim();
if (!cleanName) {
  throw new Error("displayName is required");
}
```

```
test("rejects empty displayName", () => {
  assert.throws(() => createProfile(" "), /required/);
});
```

```
npm test
git add src/profile.js test/profile.test.js
git commit -m "feat: validate required display name"
```

Push dan buka Pull Request

```
git push -u origin feature/APP-101-editable-profile
```

```
Title   : APP-101: add editable user profile validation  
Summary : normalize + validate displayName/bio, add tests  
Test    : npm test; " Raka " -> "Raka"; nama kosong -> error  
Closes APP-101
```

Sebelum minta review, lakukan self-review di tab Files changed: cari debug code, file nyasar, secret, atau perubahan di luar scope.

BAB 4 – Review, CI, dan Revisi

Checks otomatis

```
Pull Request #14  
[PASS] ci / test  
[WAIT] review
```

CI merah? Jangan minta reviewer mengabaikannya. Baca log, perbaiki di branch yang sama, push – PR terupdate otomatis.

Request changes (Akun A)

```
src/profile.js  
"displayName.trim() error bila input bukan string.  
Tangani input invalid secara eksplisit + test."
```

```
Review result: REQUEST CHANGES
```

Request changes itu normal, bukan kegagalan. Jangan buat PR baru – perbaiki di branch yang sama.

Menangani review (Akun B)

```
function createProfile(displayName, bio = "") {  
  if (typeof displayName !== "string") {  
    throw new Error("displayName must be a string");  
  }  
  const cleanName = displayName.trim();  
  const cleanBio = typeof bio === "string" ? bio.trim() : "";  
  if (!cleanName) {  
    throw new Error("displayName is required");  
  }  
  return { displayName: cleanName, bio: cleanBio };  
}
```

```
npm test  
git add src/profile.js test/profile.test.js  
git commit -m "fix: handle invalid profile input"  
git push
```

Balas komentar review dengan singkat, misalnya "Handled invalid input + added coverage", lalu minta re-review.

BAB 5 – Konflik dan Integrasi

Main bergerak saat PR direview

```
A---B---E---F main          (maju duluan)
      \---C---D feature/APP-101 (tertinggal)
```

```
git fetch origin
git log --oneline --graph --decorate --all
```

Merge vs rebase

Cara	Efek	Catatan
git merge origin/main	History asli + merge commit	Aman untuk branch yang sudah dipush
git rebase origin/main	History linear, commit ditulis ulang	Butuh force push; ikuti policy tim

Membaca conflict marker

```
<<<<<< HEAD
  return { displayName: cleanName, bio: cleanBio };
=====
  return { displayName: cleanName, bio: cleanBio,
            updatedAt: Date.now() };
>>>>>> origin/main
```

- Saat MERGE: HEAD = branch feature-mu; sisi bawah = origin/main.
- Saat REBASE: terbalik - HEAD = main (base baru), sisi bawah = commit feature-mu yang sedang direplay.
- Jangan asal hapus marker: pahami maksud kedua sisi, gabungkan, lalu test ulang.

Menyelesaikan

```
// hasil gabungan kedua maksud
return { displayName: cleanName, bio: cleanBio,
        updatedAt: Date.now() };
```

```
npm test
# jika merge:
git add src/profile.js && git commit && git push
# jika rebase:
git add src/profile.js && git rebase --continue
git push --force-with-lease --force-if-includes
```

Jangan pernah git push --force polos. --force-with-lease menolak push bila remote berubah tanpa sepengetahuanmu.

BAB 6 – Squash Merge, Sinkronisasi, dan Hotfix

Squash and merge (Akun A)

```
feature: 3 commit kecil + resolusi conflict  
main   : 1 commit ->  
"APP-101: add editable user profile validation (#14)"
```

Syarat merge: checks hijau, approval lengkap, conversations resolved, conflict selesai.

Setelah merge (Akun B)

```
git switch main && git pull origin main  
git branch -d feature/APP-101-editable-profile  
git push origin --delete feature/APP-101-editable-profile
```

Hotfix APP-102 – batasi displayName 50 karakter

```
git switch main && git pull origin main  
git switch -c fix/APP-102-display-name-length
```

```
const MAX = 50;  
if (cleanName.length > MAX) {  
  throw new Error(  
    `displayName must be at most ${MAX} characters`  
  );  
}
```

```
test("boundary 50/51", () => {  
  assert.ok(createProfile("a".repeat(50)));  
  assert.throws(() => createProfile("a".repeat(51)));  
});
```

```
npm test  
git add src/profile.js test/profile.test.js  
git commit -m "fix: limit profile display name length"  
git push -u origin fix/APP-102-display-name-length
```

Buka PR terpisah. Hotfix harus kecil, satu scope, mudah direvert. Cek boundary: 50 diterima, 51 ditolak.

BAB 7 – Simulasi 3 Akun: Parallel Work

Mode ini melatih hal yang tidak bisa disimulasikan 2 akun: dua developer bekerja bersamaan di file yang sama, review silang, dan konflik yang muncul alami.

Setup (Akun A)

- Tambahkan B dan C sebagai collaborator; protection: PR + 1 approval + CI.
- Buat dua ticket yang sengaja menyentuh src/profile.js:

```
APP-103 (B): tambah field opsional "location" (trim, default "")
APP-104 (C): tambah formatProfile(profile) -> "name - bio"
```

Naskah role-play (B dan C paralel)

#	Akun	Aksi
1	B + C	Pull main, buat branch masing-masing, coding + test (bersamaan, jangan saling tunggu)
2	B + C	Push dan buka PR masing-masing
3	B <-> C	Review silang: B me-review PR C, C me-review PR B
4	A	Approve + squash merge PR yang siap duluan (misal APP-103)
5	C	PR-nya tertinggal: fetch + merge origin/main, resolve conflict, test, push
6	A	Review ulang, approve, squash merge APP-104
7	B, C	Sync main, hapus branch

Konflik alami

Keduanya mengubah baris `module.exports`, jadi developer yang merge belakangan pasti konflik:

```
<<<<<<< HEAD
module.exports = { createProfile, formatProfile };
=====
module.exports = { createProfile };
>>>>>>> origin/main
```

```
// resolusi: pertahankan maksud kedua sisi  
module.exports = { createProfile, formatProfile };
```

Aturan mode paralel

- Jangan push ke branch orang lain; beri feedback lewat komentar PR.
- Yang merge belakangan wajib integrasi + test ulang.
- Konflik besar berulang = tanda task perlu dipecah atau butuh koordinasi sebelum coding.

BAB 8 – Simulasi 4 Akun: Dua Reviewer

Mode paling mendekati perusahaan: satu author, dua reviewer wajib, dan maintainer yang mengeksekusi merge.

Role

Akun	Role
A	Tech Lead / Maintainer – aturan, keputusan akhir, merge
B	Reviewer 1 (senior dev)
C	Reviewer 2 (senior dev)
D	Developer – author PR

Setup tambahan (Akun A)

- Require approvals: 2.
- Aktifkan "dismiss stale approvals": push baru menggugurkan approval lama.
- CODEOWNERS agar B dan C otomatis diminta review:

```
# .github/CODEOWNERS
src/ @akun-b @akun-c
```

Naskah role-play

#	Akun	Aksi
1	A	Buat ticket APP-105: fix empty bio handling; assign ke D
2	D	Branch fix/APP-105-empty-bio, coding + test, push, buka PR
3	B	Review: REQUEST CHANGES (minta test tambahan)
4	C	Review: APPROVE – dua reviewer boleh beda pendapat
5	D	Perbaiki di branch yang sama, push
6	–	Approval C otomatis gugur (stale) karena ada commit baru
7	B	Re-review: APPROVE

#	Akun	Aksi
8	C	Re-review: APPROVE (2/2 lengkap)
9	A	Cek checks + conversations, lalu squash merge
10	D	Sync main, hapus branch

Checks [PASS] ci / test
 Reviews [PASS] 2/2 approvals (B, C)
 Status READY TO MERGE

Dinamika dua reviewer

- Reviewer beda pendapat? Diskusikan di thread PR sampai ada keputusan, bukan lewat jalur pribadi. Tech Lead jadi pemutus bila buntu.
- Author tidak pernah meng-approve PR-nya sendiri.
- 2 approval bukan auto-merge – merge tetap keputusan maintainer.
- Commit baru = review ulang. Itu sebabnya PR kecil lebih cepat masuk.

BAB 9 – Cherry-pick

Apa itu

Cherry-pick menyalin SATU commit tertentu ke branch lain sebagai commit baru (SHA berbeda, isi perubahan sama). Berguna saat kamu butuh satu perbaikan tanpa membawa seluruh branch.

Kapan dipakai

- Membawa hotfix dari main ke release branch (kasus paling umum).
- Menyelamatkan commit yang terlanjur dibuat di branch yang salah.
- Mengambil sebagian kecil pekerjaan tanpa merge semuanya.

Command inti

```
git log --oneline main          # cari SHA commit target
git switch release/1.0
git cherry-pick -x <sha>        # -x catat asal commit
git cherry-pick -x <sha1> <sha2> # beberapa sekaligus
git cherry-pick --continue      # setelah resolve conflict
git cherry-pick --abort         # batalkan
```

Simulasi: bawa hotfix APP-102 ke release/1.0

```
# Akun A membuat release branch (sekali saja)
git switch main && git pull
git switch -c release/1.0 && git push -u origin release/1.0

# Developer membawa hotfix
git switch main && git pull origin main
git log --oneline -5 # catat SHA commit hotfix APP-102
git switch release/1.0
git pull origin release/1.0
git cherry-pick -x <sha-hotfix>
npm test
git push origin release/1.0
# atau lewat PR bila release branch juga protected
```

Konflik saat cherry-pick diselesaikan persis seperti biasa: edit file, git add, lalu git cherry-pick --continue.

Aturan keselamatan

- Cherry-pick = duplikasi commit; bukan pengganti merge untuk pekerjaan besar.
- Selalu pakai -x agar jejak asal commit tercatat.
- Test ulang setelah pick - konteks branch target bisa berbeda.

- Rantai cherry-pick panjang antar branch aktif = resep konflik; pertimbangkan merge.

BAB 10 – Penutup

Workflow ringkas

```
MULAI : git switch main -> git pull -> switch -c feature/T-x
KERJA : edit -> test -> status/diff -> add <file> -> commit
KIRIM : git push -u origin <branch> -> buka PR -> self-review
REVIEW : request changes? perbaiki branch yang SAMA -> push
MERGE : checks hijau + approval (mode 4: dua) -> squash merge
SELESAI: sync main -> hapus branch
        (perlu di release? cherry-pick -x)
```

Kesalahan yang harus dihindari

Kesalahan	Perbaikan
Coding langsung di main	Branch per task
git add . tanpa lihat diff	Stage hanya file relevan
PR baru saat request changes	Perbaiki branch yang sama
Asal pilih sisi conflict	Pahami dua maksud, gabungkan, test
Force push polos	--force-with-lease sesuai policy
Merge saat CI merah	Perbaiki checks dulu
Approve PR sendiri	Review silang / tunggu reviewer
Cherry-pick tanpa -x	Jejak asal commit hilang

Kriteria lulus

- [] Satu siklus penuh APP-101: branch -> PR -> review -> squash merge
- [] Menangani request changes tanpa PR baru
- [] Menyelesaikan conflict merge DAN rebase, lalu test ulang
- [] Mode 3 akun: dua PR paralel + review silang + konflik alami selesai
- [] Mode 4 akun: 2 approval, paham stale review, merge oleh maintainer
- [] Cherry-pick hotfix ke release branch dengan -x

Kalau semua tercentang dan kamu bisa menjelaskan KENAPA setiap langkah dilakukan – bukan sekadar hafal – kamu siap masuk ke tim software sungguhan. Ulangi simulasi dengan rotasi role sampai semua posisi pernah kamu rasakan.